RABBIT-HOLE, CONTINUED

X_eTaL and Mojo on the Adreno 702

One day, one dev board: array code from two languages, running on the GPU of an Arduino UNO Q and checked against an independent oracle.

Mike Wright

Software Wrighter Lab

2026-10-23

THE BOARD

A 2 GB Arduino with a GPU nobody targets

Arduino UNO Q

- Qualcomm Dragonwing QRB2210
- Four Cortex-A53 cores, reported as Kryo-V2 at 2016 MHz
- Qualcomm Adreno 702 GPU
- 1.7 GiB usable RAM, 870 MiB swap
- Debian 13.1 aarch64, glibc 2.41, Mesa 25.2.6

Why it is interesting

- Mojo's GPU support lists NVIDIA, AMD and Apple. No Qualcomm, no OpenCL, no Vulkan.
- Mesa exposes the GPU twice: Turnip for Vulkan 1.0 and rusticl for OpenCL 3.0.
- CPU and GPU share one small memory. Every copy and every byte matters.
- Mojo's own requirements ask for 8 GB of RAM.

Source: adreno-poc/docs/uno-q-capabilities.md, collected over ssh on 2026-10-23

THE QUESTION

What does it take to run parallel code on this GPU, written in Mojo and written in X_eTaL?

Not a benchmark. A proof: the same answer the language's own evaluator gives, produced by the Adreno, with every intermediate form captured so the path can be read.

THE DAY

From plan to proof in eight hours

Time	Step	What it established
08:58	Plan	13-step saga; research notes corrected against what the repos already had
09:05	Scaffold	Gate, board inventory script, host setup that stays inside the project
11:44	Host Vulkan	Vector add on an RTX 5060 Ti, every element checked, 1 K to 1 M
13:54	Inventory	Physical Adreno in both Turnip and rusticl; OpenCL first
14:06	X_eTaL on Adreno	Five source demos, 71 outputs, all agree with the evaluator
14:24	Mojo host on Adreno	Cross-compiled Mojo drives the same kernel, 24 of 24 outputs agree
16:49	Mojo kernel, 3 GPUs	One Mojo kernel on M1 Max, RTX 5060 Ti and, via SPIR-V, the Adreno

Two coding agents under agentrail: one on the Mac wrote the plan, one on an Arch Linux box with the board on ssh did the work. Every step left an evidence directory and a handoff document.

Times are commit times, Pacific, on the branch research-qualcomm-adreno-702-gpu

INVENTORY

The GPU is real, and it answers to two names

Vulkan: Turnip Adreno (TM) 702

Property	Value
API version	1.0.318
Queue family 0	graphics, compute, transfer
Max work-group invocations	512
Shared memory	16 KiB
Also listed	llvmpipe, a CPU renderer

OpenCL: rusticl FD702, Qualcomm

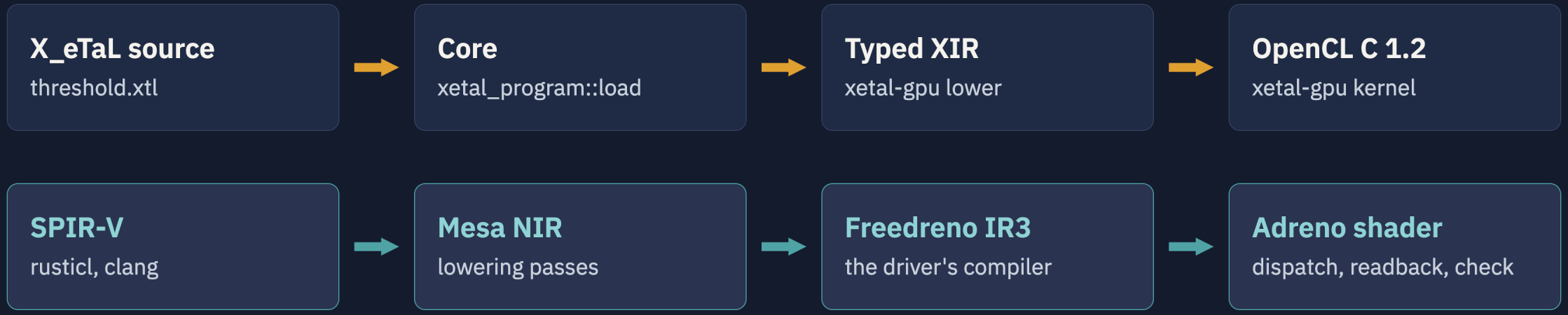
Property	Value
Platform version	OpenCL 3.0
Kernel language	OpenCL C 1.2
Max work-group size	512
Double precision	none
Device type	GPU, the only device

Decision: OpenCL first. X_eTaL-gpu already emits OpenCL C 1.2, so the existing path could be tried with no new code. Turnip reports Vulkan 1.0 and the host demo required 1.1, so Vulkan waits for a compatibility change. A run on llvmpipe never counts.

vulkaninfo and clinfo on the board; adreno-poc/docs/evidence/unoq-inventory-20261023

PATH ONE: X_ETAL

Source to silicon, nothing by hand



The top row is the X_eTaL ecosystem, unchanged. The bottom row is Mesa on the board. Every box's output was captured as text and committed as evidence.

The pinned X_eTaL-gpu was cross-built on Arch and lowered the programs on the board itself

THE SOURCE

Eight lines of X_eTaL, as typed and as rendered

As typed: plain ASCII

```
"ac:" u_se< "Accel"
x := ((r_ange 8) / 4) - 1.0
x
0.5 ac:t_hreshold x
ac:r_elu x
n := (r_ange 8) - 4
ac:r_elu n
(n m_od 2) ac:k_eep n
```

An underscore underlines the letter before it and makes the name a function. A library alias before a colon becomes a superscript.

As rendered: the editor's decorated pane

```
"ac:" use< "Accel"
x ← ((range 8) ÷ 4) - 1.0
x
0.5 ᵃᶜthreshold x
ᵃᶜrelu x
n ← (range 8) - 4
ᵃᶜrelu n
(n mod 2) ᵃᶜkeep n
```

Blue: built-in function. Teal: library function, with its alias raised. Amber: number, string, or the import macro. Light blue: operator. The evaluator prints five lines; these are the oracle.

adreno-poc/docs/evidence/unoq-xetal-opencl-20261023/threshold.xtl; the text is the output of xetal render, set in JuliaMono, the editor's colors

IR AND KERNEL

Typed XIR, then one fused OpenCL kernel

threshold.xir, the Float path

```
%l = const f64 [8] 1.0 2.0 ... 8.0
%r = const f64 [] 4.0
%div = map div %l %r
%sub = map sub %div %r_2
%gt = map gt %sub %r_3
%f = cast f64 %gt
%mul = map mul %sub %f
%max = map max %sub %r_4
```

Library calls inlined, the range folded to a constant, every array operation one typed node. The comparison yields a Bool, the cast makes a mask, the multiply is the threshold, the max is ReLU.

threshold.cl, generated

```
__kernel void k0(__global const float* b_l, ...) {

size_t i = get_global_id(0);
if (i >= 8) return;
float t_div = (b_l[i] / 4.0f);
float t_sub = (t_div - 1.0f);
int t_gt = (t_sub > 0.5f);
float t_f = (float)t_gt;
float t_mul = (t_sub * t_f);
float t_max = fmax(t_sub, 0.0f);
...
```

Consecutive maps of one shape fuse into one kernel; the IR temporaries become register expressions. The guard matters because the launch rounds up to a work-group. Floats are f32 on this no-fp64 device, Ints i64.

xetal-gpu lower and xetal-gpu kernel, run on the board; the full files are in the evidence directory

X_ETAL RESULTS

Five programs, 71 outputs, every one agrees

Program	Outputs checked	Coverage	Result
threshold.xtl	5 vectors, 40 elements	Threshold, Float and Int ReLU, mask selection	Pass
vector-add.xtl	2 vectors, 16 elements	Int and Float vector addition and scaling	Pass
saxpy.xtl	1 vector, 8 elements	Fused scale and add	Pass
reduce-sum.xtl	5 scalars	Sum, maximum, dot product, Int and Float reductions	Pass
pipeline.xtl	2 scalars	Multiply, add, select, reduce	Pass

How they were checked

Ints exactly, Floats to a relative 1e-5, every element and not a sample, against the evaluator baselines copied unchanged from X_eTaL-gpu's regression tests. The device selector refuses anything that is not the one Qualcomm GPU.

The one board-specific change

The default 256-item work-group passed the maps and failed the reductions: the compiled reduce_max_long kernel allows at most 192 work-items on this GPU. A schedule file with work_group = 128 fixed it. No emitter or runtime change.

RESULT: 5 of 5 X_eTaL source demos agree with evaluator baselines on physical FD702

TO THE METAL

The Adreno instructions, read against the source

Freedreno IR3 instruction	Role in the kernel
(sy)mul.f r3.w, r3.w, c16.y	Multiply by 0.25: the source's divide by four
add.f r3.w, r3.w, c16.z	Add -1.0 to form x
cmps.f.lt hr3.z, (0.5), r3.w	Test 0.5 < x, that is x > 0.5
cov.u16f32 (even)r2.x, hr3.z	Convert the comparison into a Float mask
mul.f r2.x, r3.w, r2.x	x times the mask: the threshold
max.f r3.w, r3.w, (0.0)	ReLU
stg.u32 g[r5.x], r2.x, 1	Store one threshold element
stg.u32 g[r2.w], r3.w, 1	Store one ReLU element

Captured with Mesa's own switches: RUSTICL_DEBUG=nir for every NIR stage and IR3_SHADER_DEBUG=disasm for the native shader. The driver's binary container holds SPIR-V 1.4 with OpenCL physical pointers, not machine code; the Adreno instructions appear only in this diagnostic compile. The 64-bit Int remainder expands to dozens of 32-bit instructions around this short Float sequence.

adreno-poc/docs/evidence/unoq-deep-dive-20261023: xetal-native.disasm, xetal-mesa-debug.log, threshold.opencl.spvasm

PATH TWO: MOJO

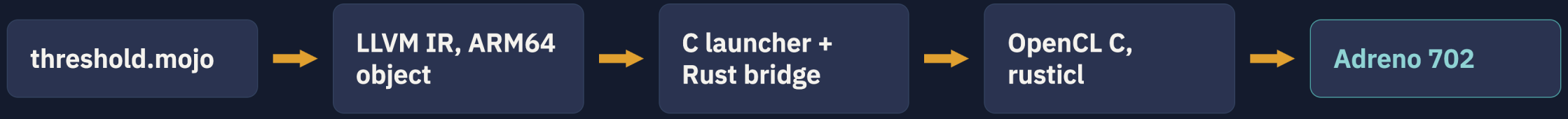
Mojo has no Adreno target, so Mojo takes the host's seat

Three walls

- No Qualcomm, OpenCL or Vulkan target: Mojo's runtime looks for CUDA, HIP and Metal only.
- No compiler on the board: 1.7 GiB of RAM against a documented 8 GB, and no pip in the image.
- A normal cross-built Mojo executable died with SIGILL: the runtime library uses LSE atomics the Cortex-A53 lacks.

Three doors

- Cross-compile on Arch to an object: --emit object, target aarch64-unknown-linux-gnu, CPU cortex-a53.
- Export one C-ABI function from Mojo; a 30-line C launcher provides storage, so no Mojo runtime is linked.
- A Rust bridge with a C ABI hands the work to the same XIR, OpenCL emitter and runtime X_eTaL used.



The executable needs only libc and the bridge library; the SIGILL diagnosis is a likely cause, not a traced fault address

THE MOJO

One exported function: fill, call, check

```
@export("mojo_adreno_demo")
def mojo_adreno_demo(inputs, outputs, expected: Pointer[Float32, MutAnyOrigin],

n: Int) abi("C") -> Int32:
for i in range(n):
var x = Float32(i + 1) / 4.0 - 1.0
inputs[unsafe_offset=i] = x
expected[unsafe_offset=i] = x
var mask = Float32(1.0) if x > 0.5 else Float32(0.0)
expected[unsafe_offset=n + i] = x * mask
expected[unsafe_offset=2 * n + i] = max(x, Float32(0.0))
var status = external_call["adreno_threshold_v1", Int32](inputs, outputs, n, 3 * n)

if status != 0: return status
var failures: Int32 = 0
for i in range(3 * n):
if outputs[unsafe_offset=i] != expected[unsafe_offset=i]: failures += 1

return failures
```

What Mojo does

Builds the same eight inputs as the X_eTaL program, computes its own threshold and ReLU reference on the CPU, calls the bridge, and counts mismatches over all 24 returned values.

What the bridge does

Verifies there is exactly one Qualcomm GPU, builds a checked XIR program with a dynamic input, and runs it through X_eTaL-gpu's planner and OpenCL runtime.

What it compiles to

An ELF64 AArch64 object with one unresolved symbol, adreno_threshold_v1, linked by the ARM64 GCC with the launcher. The disassembly shows fcmp, fcsel and fmul for the reference, then bl to the bridge.

adreno-poc/mojo/threshold.mojo, condensed for the slide; Mojo 1.2.0.dev2026101305, pinned

MOJO RESULTS

24 of 24 outputs agree, and a planted error is caught

```
Mojo bridge: opencl:0 FD702 (GPU, Qualcomm; rusticl)
OpenCL device kernels: 0.003692 ms; total 26.954 ms
-0.75 -0.5 -0.25 0 0.25 0.5 0.75 1
-0 -0 -0 0 0 0 0.75 1
0 0 0 0 0.25 0.5 0.75 1
ok: 3 lines agree (tolerance 1e-05)
PASS: deliberately corrupted final output was rejected
RESULT: 24 of 24 outputs agree with X_eTaL baseline
and Mojo reference on physical FD702
```

2 oracles

Mojo's own CPU reference and X_eTaL's saved evaluator output, both exact on these values.

1 negative test

The launcher's --corrupt-output flag adds one to element 23; the run must fail, and did: "Mismatch at 23: got 2 expected 1".

The 27 ms total is one-shot setup, upload, dispatch and download for eight elements. It is a proof of execution, not a throughput number, and nobody should quote it as one.

adreno-poc/docs/evidence/unoq-mojo-opencl-20261023: demo.log, corrupt.log, environment.log with binary hashes

PATH THREE: A MOJO KERNEL

The same two activations, in each language

X_eTaL: the Accel library, and the call

```
⍝ libs/Accel/src/Accel.xtl
⁾relu ← { x → x max 0 }
⁾threshold ← { t x → x × float x > t }
⍝ threshold.xtl
x ← ((range 8) ÷ 4) - 1.0
0.5 ᵃᶜthreshold x
ᵃᶜrelu x
```

Whole-array functions. No threads, no indices, no buffers: the compiler finds the parallelism and writes the kernel.

Mojo: one kernel, one thread per element

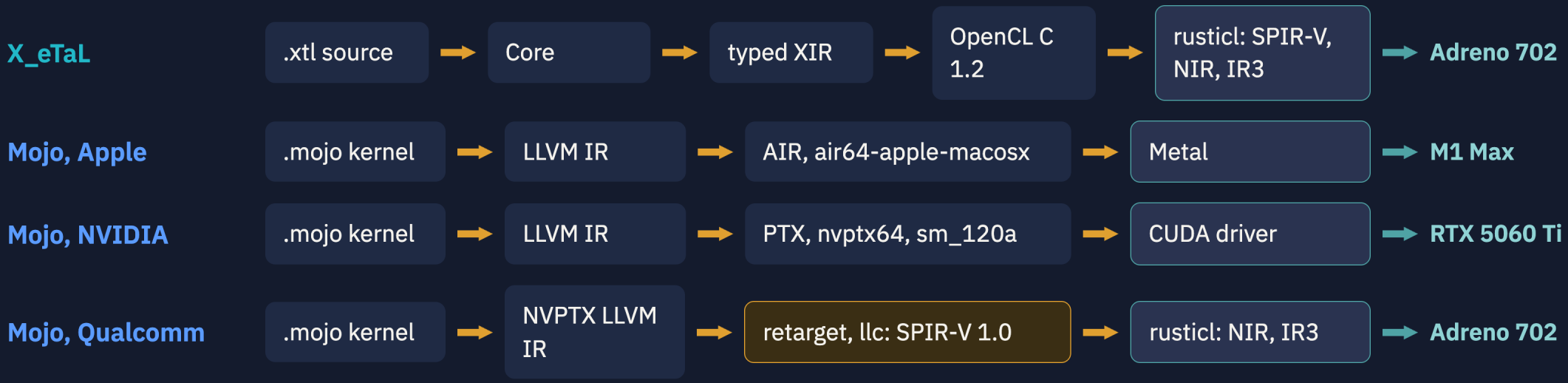
```
# mojo/activation_gpu.mojo
def activations(x, threshold, relu: Pointer[...],
t: Float32, n: Int32):
var i = Int(block_idx.x * block_dim.x
+ thread_idx.x)
if i < Int(n):
var v = x[unsafe_offset=i]
var mask = 1.0 if v > t else 0.0
threshold[unsafe_offset=i] = v * mask
relu[unsafe_offset=i] = max(v, 0.0)
```

Written as for Apple or NVIDIA. The host opens a DeviceContext, copies x, launches with grid and block sizes, copies back.

Same formula three ways: x × float(x > t) in X_eTaL, v * mask in Mojo, gt, cast, mul in XIR. Mojo types shortened to fit.

EVERY STEP

Two sources, four roads, three GPUs



Dark boxes are the language's own compiler. Teal boxes are the vendor's or Mesa's driver. The amber box is the one piece added today: a tested script that rewrites Mojo's NVIDIA IR for OpenCL, then LLVM 23's SPIR-V back-end.

X_eTaL-gpu's OpenCL road also runs on the M1 Max and NVIDIA cards; this deck shows it on the Adreno

THE RETARGET

Seven substitutions turn NVIDIA IR into OpenCL SPIR-V

Mojo's NVPTX LLVM IR	OpenCL SPIR-V form
nvptx64-nvidia-cuda	spirv64v1.0
ptx_kernel	spir_kernel
plain ptr buffers	ptr addrspace(1)
sreg.ctaid.x	get_group_id(0)
sreg.ntid.x	get_local_size(0)
sreg.tid.x	get_local_id(0)
sm_120a target attributes	removed

The arithmetic Mojo emitted, fcmp, select, fmul and maxnum, passes through untouched. Anything else, barriers, shared memory, atomics, library calls, is refused rather than guessed.

```
llc -mtriple=spirv64v1.0-unknown-unknown \

-filetype=obj act.bc -o act.spv
spirv-val --target-env opencl2.1 act.spv
```

Why a script at all

Mojo's compiler refuses a SPIR-V target: "target spirv64-unknown-unknown is not supported by this build". Its NVIDIA output is plain LLVM, and Arch's LLVM 23 has the SPIR-V back-end.

One surprise

The unversioned target emitted SPIR-V 1.4, which the OpenCL validation rejected. Pinning SPIR-V 1.0 validated and ran.

scripts/nvptx-to-spirv.py with a golden-file test; loaded on the board with clCreateProgramWithIL, which rusticl supports

THREE GPUS

One Mojo kernel, three vendors, 24 of 24 each

Apple M1 Max

Metal, through DeviceContext, as Modular ships it

24 of 24 agree

NVIDIA RTX 5060 Ti

CUDA, through DeviceContext, the same file unchanged

24 of 24 agree

Qualcomm Adreno 702

The same kernel as SPIR-V through rusticl, from a Mojo host on the A53

24 of 24 agree, both oracles

```
input -0.75 -0.5 -0.25 0.0 0.25 0.5 0.75 1.0
threshold -0.0 -0.0 -0.0 0.0 0.0 0.0 0.75 1.0
relu 0.0 0.0 0.0 0.0 0.25 0.5 0.75 1.0
```

Identical on all three, and identical to X_eTaL's evaluator, signed zeros included.

On the Adreno also: n of 1, 65 and 129 across groups, a corrupted output rejected, an invalid module rejected.

adreno-poc/docs/mojo-on-3-gpus.md; evidence in m1max-mojo-targets-20261023 and unoq-mojo-spirv-20261023

BOTH LANGUAGES, SAME METAL

The Adreno instructions from X_eTaL and from Mojo

Step	From X_eTaL, via OpenCL C	From Mojo, via SPIR-V
compare	cmps.f.lt hr3.z, (0.5), r3.w	cmps.f.lt hr5.z, c9.z, r1.z
mask	cov.u16f32 (even)r2.x, hr3.z	cov.u16f32 (even)r0.y, hr5.z
threshold	mul.f r2.x, r3.w, r2.x	mul.f r0.y, r1.z, r0.y
relu	max.f r3.w, r3.w, (0.0)	max.f r1.z, r1.z, (0.0)
store	stg.u32 g[r5.x], r2.x, 1	stg.u32 g[r2.y], r0.y, 1
store	stg.u32 g[r2.w], r3.w, 1	stg.u32 g[r0.z], r1.z, 1

The same five operations, register names aside. One visible difference: X_eTaL folds the threshold 0.5 into the instruction as an immediate, while Mojo passes t as a kernel argument, so the Adreno reads it from constant register c9.z.

Mesa IR3_SHADER_DEBUG=disasm: xetal-native.disasm and unoq-mojo-spirv-20261023/debug/native.disasm; excerpts, scheduling flags omitted

THE BOUNDARY

What the day established, and what it did not

Established

- X_eTaL source, lowered by its own tool, runs on the physical Adreno 702: 71 of 71 outputs match the evaluator.
- A kernel written in ordinary Mojo runs on Apple, NVIDIA and the Adreno, 24 of 24 on each, from one source.
- On the Adreno, both languages reach the same Freedreno instructions for the activation.
- Every form is captured as text, and nothing was installed on the board.

Not established

- Native Adreno support in Mojo. DeviceContext cannot see the GPU; a C bridge launches the SPIR-V.
- Kernels beyond this scalar subset. Barriers, shared memory, atomics and math library calls are refused, not translated.
- Any speedup. Eight elements prove execution and agreement, not throughput.
- A Vulkan dispatch on the Adreno, or Mojo's own runtime on the A53.

Each handoff document ends with a section titled "What this does not establish"

NEXT

From one kernel to real workloads

Mojo

- Widen the retargeter one construct at a time: barriers, shared memory, math functions
- Run rust-demo's reductions and softmax on the Adreno, each against the evaluator
- Take the evidence to Modular: a SPIR-V target and an OpenCL runtime are theirs to add

Vulkan

- A Vulkan 1.0 host for Turnip, then the first dispatch on the Adreno
- XIR to GLSL compute: the same programs as Vulkan SPIR-V
- The Vulkan runtime as an X_eTaL extension beside the OpenCL one

Measurement

- Threshold, SAXPY, ReLU and softmax from 1 K to 1 M elements
- Cortex-A53 against Adreno: where the GPU starts to pay
- Whether the 4 GB board is worth buying

The order is deliberate: correctness on every road first, then size, then speed.

adreno-poc/docs/plan.md and the agentrail saga adreno-702

WHERE IT ALL IS

Everything on this deck is a text file in a public branch

What	Where
The branch	github.com/softwarewrighter/modular-fork, research-qualcomm-adreno-702-gpu, directory adreno-poc
The annotated walkthrough	adreno-poc/docs/adreno-deep-dive.md
Board inventory and decision	adreno-poc/docs/uno-q-capabilities.md
X_eTaL run, Mojo run, native dumps	adreno-poc/docs/evidence/unoq-*-20261023
The languages	github.com/softwarewrighter/X_eTaL, X_eTaL-gpu, X_eTaL-extensions; github.com/modular/modular
The earlier deep dive, CUDA, Metal, AMD, arm64	blog.softwarewrighter.com, 2026-10-21, Rabbit-hole #8